

School of Computer Science and Artificial Intelligence

Lab Assignment # 15.2

Program	: B. Tech (CSE)
Specialization	: -
Course Title	: AI Assisted Coding
Course Code	: 23CS002PC304
Semester	II
Academic Session	: 2025-2026
Name of Student	: K. Deepak
Enrollment No.	: 2303A51T02
Batch No	: 52
Date	: 17/03/26

Task 1: Setup Fast-API Backend (VS Code)**Install:**

Prompt: Create a basic FastAPI app with one GET endpoint "/" that returns a welcome message.

Code:

```
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
async def read_root():
    return {"message": "Welcome to the FastAPI app!"}
```

Run: python -m uvicorn main:app --reload

Output: <http://127.0.0.1:8000/>

```
Pretty-print ☐
{"message": "Welcome to the FastAPI app!"}
```

Explanation: This code sets up a basic FastAPI application with endpoints to create, read, update, and delete items in a list, along with docstrings for Swagger documentation.

Task 2: API – Create and Read Items

Prompt: Add GET `"/items"` and POST `"/items"` endpoints in FastAPI using a list to store items.

Code:

```
items = []
@app.get("/items")
async def read_items():
    return items
@app.post("/items")
async def create_item(item: dict):
    items.append(item)
    return {"message": "Item created", "item": item}
```

Output:

FastAPI 0.10.0 OAS 3.1
/openapi.json

default

GET / Read Root

GET /items Read Items

POST /items Create Item

Schemas

HTTPValidationError Expand all object

ValidationError Expand all object

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/items' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{"name":"Pen","price":20}'
```

Request URL

http://127.0.0.1:8000/items

Server response

Code Details

200

Response body

```
{
  "message": "Item created",
  "item": {
    "name": "Pen",
    "price": 20
  }
}
```

Download

Explanation: This code defines two endpoints in a FastAPI application to manage a list of items, allowing users to retrieve all items and add new items to the list.

Task 3: Update Existing Item

Prompt: Add a PUT `"/items/{index}"` endpoint to update an item in a list by index.

Code:

```
@app.put("/items/{index}")
async def update_item(index: int, item: dict):
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    items[index] = item
    return {"message": "Item updated", "item": item}
```

Output:

The screenshot displays the FastAPI Swagger UI interface. At the top, it shows 'FastAPI 0.9.0' and 'GAS 3.1'. Below this, a list of endpoints is shown under the 'default' section:

- GET / Read Root
- GET /items Read Items
- POST /items Create Item
- PUT /items/{index} Update Item

Below the endpoints, there are two schema sections:

- HTTPValidationError - Expanded all - object
- ValidationError - Expanded all - object

The 'Curl' section shows a successful PUT request:

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/items/0' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Pencil",
    "price": 10
  }'
```

The 'Request URL' is `http://127.0.0.1:8000/items/0`.

The 'Server response' section shows a 200 status code. The 'Response body' is:

```
{
  "message": "Item updated",
  "item": {
    "name": "Pencil",
    "price": 10
  }
}
```

The 'Response headers' are:

```
content-length: 62
content-type: application/json
date: Tue, 17 Mar 2025 11:01:32 GMT
server: uvicorn
```

The 'Responses' section shows a table with columns 'Code' and 'Description'.

Explanation: This code adds a PUT endpoint to the FastAPI application that allows users to update an existing item in the list by specifying its index, with error handling for out-of-range indices.

Task 4: Delete Item

Prompt: Add a DELETE `"/items/{index}"` endpoint to remove an item from a list.

Code:

```
@app.delete("/items/{index}")
async def delete_item(index: int):
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    removed_item = items.pop(index)
    return {"message": "Item deleted", "item": removed_item}
```

Output:

The screenshot displays a REST client interface with the following sections:

- Curl:** `curl -X 'DELETE' \ 'http://127.0.0.1:8000/items/0' \ -H 'accept: application/json'`
- Request URL:** `http://127.0.0.1:8000/items/0`
- Server response:**

Code	Details
200	Response body <pre>{ "message": "Item deleted", "item": { "name": "Pen", "price": 20 } }</pre> Response headers <pre>content-length: 59 content-type: application/json date: Tue, 17 Mar 2026 11:04:19 GMT server: uvicorn</pre>
- Responses:** A table with columns 'Code' and 'Description' is partially visible at the bottom.

Explanation: This code adds a DELETE endpoint to the FastAPI application that allows users to remove an item from the list by specifying its index, with error handling for out-of-range indices.

Task 5: Auto-Generated Documentation

Prompt: Add docstrings to all FastAPI endpoints so they appear in Swagger documentation.

Code:

```
@app.get("/items")
async def read_items():
    """
    Retrieve all items from the list.
    """
    return items

@app.post("/items")
async def create_item(item: dict):
    """
    Create a new item and add it to the list.
    """
    items.append(item)
    return {"message": "Item created", "item": item}

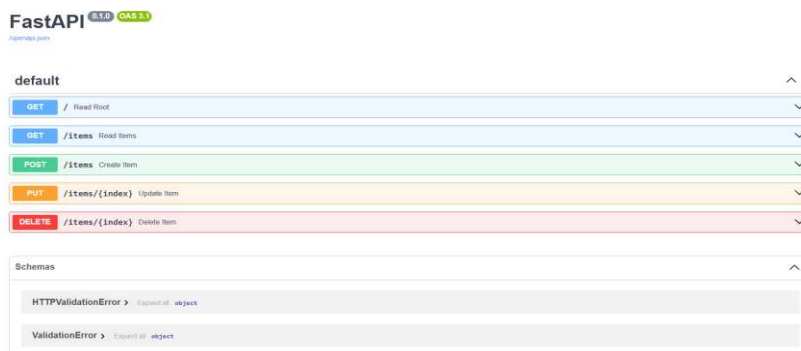
@app.put("/items/{index}")
async def update_item(index: int, item: dict):
    """
    Update an existing item in the list by its index.
    """
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    items[index] = item
    return {"message": "Item updated", "item": item}

@app.delete("/items/{index}")
async def delete_item(index: int):
    """
    Delete an item from the list by its index.
    """
    if index < 0 or index >= len(items):
        return {"error": "Index out of range"}
    removed_item = items.pop(index)
    return {"message": "Item deleted", "item": removed_item}
```

Open:

- <http://127.0.0.1:8000/docs> (Swagger UI)

Output:



Explanation: This code adds docstrings to each endpoint in the FastAPI application, providing descriptions that will be displayed in the Swagger UI for better documentation and understanding of each endpoint's functionality.